

操作系统实验报告

实验一：Linux 操作系统环境

学号	2024141460445
姓名	许盛凯

任务一：文件与目录操作

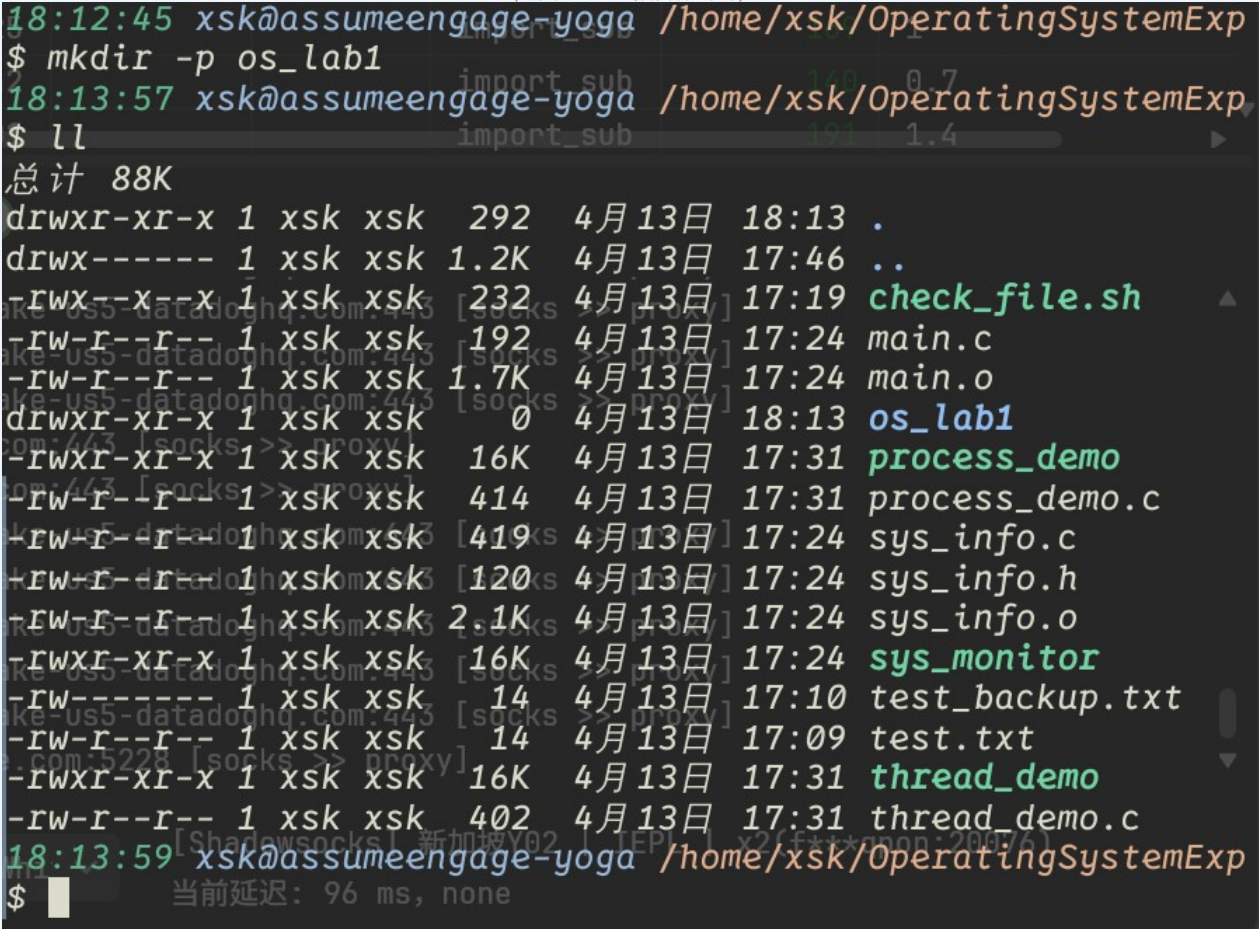
1.1 命令执行记录

(将每条命令及其执行结果截图粘贴到下方对应区域)

步骤 1 创建 os_lab1 目录

 mkdir 命令执行结果

(请在此处粘贴截图)



```
18:12:45 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ mkdir -p os_lab1
18:13:57 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ ll
总计 88K
drwxr-xr-x 1 xsk xsk 292 4月13日 18:13 .
drwx----- 1 xsk xsk 1.2K 4月13日 17:46 ..
-rwx--x--x 1 xsk xsk 232 4月13日 17:19 check_file.sh
-rw-r--r-- 1 xsk xsk 192 4月13日 17:24 main.c
-rw-r--r-- 1 xsk xsk 1.7K 4月13日 17:24 main.o
drwxr-xr-x 1 xsk xsk 0 4月13日 18:13 os_lab1
-rwxr-xr-x 1 xsk xsk 16K 4月13日 17:31 process_demo
-rw-r--r-- 1 xsk xsk 414 4月13日 17:31 process_demo.c
-rw-r--r-- 1 xsk xsk 419 4月13日 17:24 sys_info.c
-rw-r--r-- 1 xsk xsk 120 4月13日 17:24 sys_info.h
-rw-r--r-- 1 xsk xsk 2.1K 4月13日 17:24 sys_info.o
-rwxr-xr-x 1 xsk xsk 16K 4月13日 17:24 sys_monitor
-rw----- 1 xsk xsk 14 4月13日 17:10 test_backup.txt
-rw-r--r-- 1 xsk xsk 14 4月13日 17:09 test.txt
-rwxr-xr-x 1 xsk xsk 16K 4月13日 17:31 thread_demo
-rw-r--r-- 1 xsk xsk 402 4月13日 17:31 thread_demo.c
18:13:59 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$
```

步骤 2 创建 test.txt 并写入内容

 touch / echo 命令执行结果
(请在此处粘贴截图)

```
rw-r--r-- 1 xsk xsk 402 4月13日 17:31 thread_demo.c
8:14:51 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cat test
test_backup.txt test.txt
8:14:51 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cat test
test_backup.txt test.txt
8:14:51 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cat test.txt
Hello,World
8:15:01 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ echo "Hello,World" > test.txt
8:15:21 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
```

步骤 3 复制并重命名为 test_backup.txt

 cp 命令执行结果
(请在此处粘贴截图)

```

或者在本地使用 : info '(coreutils) cp invocation'
18:16:08 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cp --backup -T test
test_backup.txt test.txt
18:16:08 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cp --backup -T test
test_backup.txt test.txt
18:16:08 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cp --backup -T test
test_backup.txt test.txt
18:16:08 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cp --backup -T test.txt
check_file.sh process_demo.c test_backup.txt
main.c sys_info.c test.txt
main.o sys_info.h thread_demo
os_lab1/ sys_info.o thread_demo.c
process_demo sys_monitor
18:16:08 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cp --backup -T test.txt test
test_backup.txt test.txt
18:16:08 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cp --backup -T test.txt test
test_backup.txt test.txt
18:16:08 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cp --backup -T test.txt test_backup.txt
cp: 是否覆盖 'test_backup.txt'? y
18:16:27 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ 当前延迟: 96 ms, none

```

步骤 4 查看权限并修改为仅所有者可读写

 ls -l 及 chmod 执行结果
(请在此处粘贴截图)


```

-rwxr-xr-x 1 xsk xsk 16K 4月13日 17:24 sys_monitor
-rw-r--r-- 1 xsk xsk 12 4月13日 18:16 test_backup.txt
-rw----- 1 xsk xsk 14 4月13日 17:10 test_backup.txt~
-rw-r--r-- 1 xsk xsk 12 4月13日 18:15 test.txt
-rwxr-xr-x 1 xsk xsk 16K 4月13日 17:31 thread_demo
-rw-r--r-- 1 xsk xsk 402 4月13日 17:31 thread_demo.c
18:16:49 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ sudo chmod 600 test_backup.txt
[sudo] xsk 的密码:
18:17:19 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ ll
总计 92K
drwxr-xr-x 1 xsk xsk 324 4月13日 18:16 .
drwx----- 1 xsk xsk 1.2K 4月13日 17:46 ..
-rwx--x--x 1 xsk xsk 232 4月13日 17:19 check_file.sh
-rw-r--r-- 1 xsk xsk 192 4月13日 17:24 main.c
-rw-r--r-- 1 xsk xsk 1.7K 4月13日 17:24 main.o
drwxr-xr-x 1 xsk xsk 0 4月13日 18:13 os_lab1
-rwxr-xr-x 1 xsk xsk 16K 4月13日 17:31 process_demo
-rw-r--r-- 1 xsk xsk 414 4月13日 17:31 process_demo.c
-rw-r--r-- 1 xsk xsk 419 4月13日 17:24 sys_info.c
-rw-r--r-- 1 xsk xsk 120 4月13日 17:24 sys_info.h
-rw-r--r-- 1 xsk xsk 2.1K 4月13日 17:24 sys_info.o
-rwxr-xr-x 1 xsk xsk 16K 4月13日 17:24 sys_monitor
-rw----- 1 xsk xsk 12 4月13日 18:16 test_backup.txt
-rw----- 1 xsk xsk 14 4月13日 17:10 test_backup.txt~
-rw-r--r-- 1 xsk xsk 12 4月13日 18:15 test.txt
-rwxr-xr-x 1 xsk xsk 16K 4月13日 17:31 thread_demo
-rw-r--r-- 1 xsk xsk 402 4月13日 17:31 thread_demo.c
18:17:20 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$

```

1.2 权限分析

(结合 ls -l 的输出，解释修改前后权限字符串的变化含义)

Linux 权限分三组：所有者/同组用户/其他用户，每组有 rwx 三位

用八进制数字表示：r=4，w=2，x=1，加起来就是该组权限

600 = 110 000 000 = 所有者可读写，其他人无任何权限 → "仅所有者可读写"

任务二：Shell 脚本编写

2.1 脚本源码

(将 check_file.sh 的完整代码粘贴到下方)

```
# check_file.sh #!/bin/bash ...

#!/bin/bash

if [ -z "$1" ]; then
    echo "usage: $0 <file_name>"
    exit 1
fi

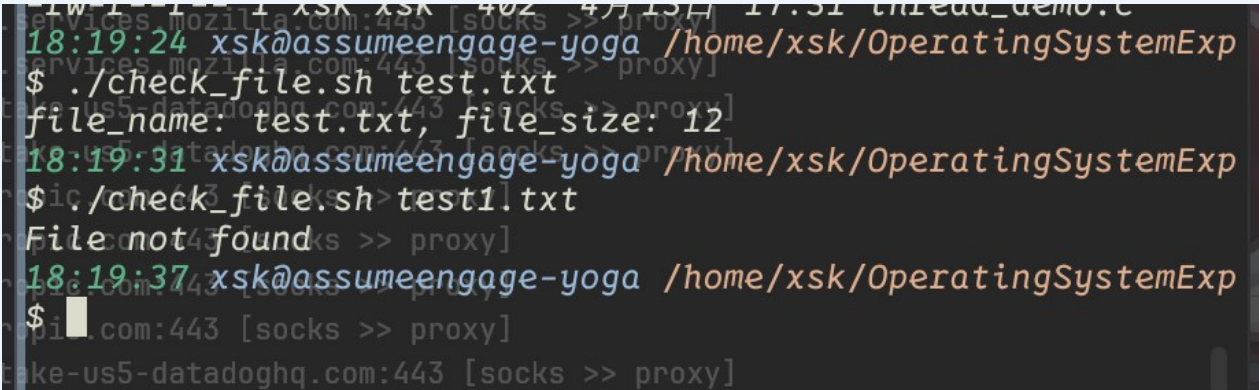
file="$1"

if [ -e "$file" ]; then
    size=$(stat -c %s "$file")
    echo "file_name: $file, file_size: $size "
else
    echo "File not found"
fi
```

2.2 运行结果

(分别测试文件存在和不存在两种情况，截图粘贴到下方)

 □□□□□□□□□□
(请在此处粘贴截图)



```
18:19:24 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ ./check_file.sh test.txt
file_name: test.txt, file_size: 12
18:19:31 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ ./check_file.sh test1.txt
File not found
18:19:37 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$
```



□□□□□□□□□□

(请在此处粘贴截图)

```
18:19:24 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ ./check_file.sh test.txt
file_name: test.txt, file_size: 12
18:19:31 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ ./check_file.sh test1.txt
File not found
18:19:37 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$
```

任务三：GCC 多文件编译与系统调用

3.1 代码补全

(将三个文件的完整源码分别填入下方)

sys_info.h

// sys_info.h ...

```
18:19:37 xsk@assumeengage-yoga /home/xsk/0
$ cat sys_info.h
#ifndef SYS_INFO_H
#define SYS_INFO_H

void print_kernel_version();
void print_username(); // 补全的声明

#endif
```

sys_info.c

// sys_info.c ...


```

18:20:05 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cat sys_info.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/utsname.h>
#include <unistd.h>
#include "sys_info.h"

void print_kernel_version() {
    struct utsname buffer;
    if (uname(&buffer) == 0) {
        printf("Kernel Version: %s\n", buffer.release);
    }
}

void print_username() {
    char *user = getenv("USER");
    if (user == NULL) user = getlogin();
    printf("Current User: %s\n", user ? user : "unknown");
}

```

main.c

// main.c ...

```

18:20:12 xsk@assumeengage-yoga /home/xsk/OperatingSystemExp
$ cat main.c
#include <stdio.h>
#include "sys_info.h"

int main() {
    printf("--- OS Lab 1: System Monitor ---\n");
    print_kernel_version();
    print_username(); // 补全的调用
    return 0;
}

```

3.2 编译过程记录

(依次执行四条编译命令，将终端输出截图粘贴到下方)

 □□□□□□□□
(请在此处粘贴截图)

```
08:20:21 xsk@assumengage-yoga /home/xsk
$ gcc -Wall -c sys_info.c -o sys_info.o
gcc -Wall -c main.c -o main.o
gcc main.o sys_info.o -o sys_monitor
./sys_monitor
--- OS Lab 1: System Monitor ---
Kernel Version: 6.19.11-arch1-1
Current User: xsk
```

3.3 最终运行结果

 ./sys_monitor □□□□□□
(请在此处粘贴截图)

```
08:20:21 xsk@assumengage-yoga /home/xsk
$ gcc -Wall -c sys_info.c -o sys_info.o
gcc -Wall -c main.c -o main.o
gcc main.o sys_info.o -o sys_monitor
./sys_monitor
--- OS Lab 1: System Monitor ---
Kernel Version: 6.19.11-arch1-1
Current User: xsk
```

任务四：多进程与多线程初步

4.1 多进程 fork()

process_demo.c 源码

```
// process_demo.c ...  
  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main() {  
    pid_t pid = fork();  
  
    if (pid < 0) {  
        perror("fork failed");  
        return 1;  
    } else if (pid == 0) {  
        printf("I am child process, PID: %d\n", getpid());  
    } else {  
        printf("I am parent process, waiting for child %d\n",  
pid);  
        wait(NULL);  
        printf("Child finished.\n");  
    }  
    return 0;  
}
```

4.2 多线程 pthread

thread_demo.c 源码

```
// thread_demo.c ...
```

```

cat thread_demo.c
#include <stdio.h>
#include <pthread.h>

void* thread_func(void* arg) {
    printf("Hello from thread! Arg: %s\n", (char*)arg);
    return NULL;
}

int main() {
    pthread_t tid;
    char* msg = "OS Lab 1";

    pthread_create(&tid, NULL, thread_func, msg); // 创建线程
    pthread_join(tid, NULL); // 等待线程结束
    printf("Main thread finished.\n");
    return 0;
}

```

运行结果

 ./thread_demo □□□□
(请在此处粘贴截图)

```

18:22:38 xsk@assumeengage-yoga /home/xsk/
$ ./thread_demo
Hello from thread! Arg: OS Lab 1
Main thread finished.

```


实验思考

思考 1 (基础)

为什么在执行 `if [ $a -eq $b ]` 时，中括号与变量之间必须有空格？

在此填写你的分析...

[本质上是一个命令（内建命令），它的参数之间必须用空格分隔，否则 shell 无法正确解析 token

思考 2 (基础)

□ GCC 生成的 `.o` 文件和最终的可执行文件有什么区别？

在此填写你的分析...

.o 是目标文件，包含编译后的机器码但符号未解析（函数地址未确定）；可执行文件是链接器将多个 .o 的符号全部解析并合并后生成的，可直接由 OS 加载运行。

进阶挑战 1 (选做)

□ `fork()` 产生的父子进程中，它们是否共享同一个全局变量？结合写时复制（Copy-on-Write）机制解释原因。

在此填写你的实验现象与分析...

不共享。`fork()` 采用写时复制（Copy-on-Write）机制，子进程拥有父进程地址空间的独立副本，任何一方修改变量都不影响另一方。